

Distributed Master-Worker System

XML-RPC Based Distributed Query System
Course: Distributed Systems | Date: May 12, 2026

1. Introduction

This report presents the design, implementation, and execution results of a Distributed Master-Worker system built using Python's built-in **XML-RPC** protocol. The system distributes a person-record dataset across two worker nodes and exposes query operations through a master node that acts as a transparent coordinator. A client interacts exclusively with the master, unaware of the underlying worker topology.

2. System Architecture

The system consists of three distinct roles communicating entirely over XML-RPC:

- **Master (master.py)** — Listens on port **23001**. Routes name-based queries to the appropriate worker and fans out location/year queries to both workers, merging the results before returning them to the client.
- **Worker-1 (worker.py am)** — Listens on port **23003**. Holds records for names in the range **A–M** (data-am.json, 14 records).
- **Worker-2 (worker.py nz)** — Listens on port **23004**. Holds records for names in the range **N–Z** (data-nz.json, 14 records).
- **Client (client.py)** — Sends three test queries to the master and prints the structured responses including partial-result flags.

Component	Port	Dataset	Role
master.py	23001	—	Query router & result aggregator
worker.py am	23003	data-am.json (14 records)	Serves names A–M
worker.py nz	23004	data-nz.json (14 records)	Serves names N–Z
client.py	—	—	Test client (3 query types)

3. Routing & Aggregation Logic

The master implements two distinct routing strategies depending on the query type:

- **getbyname(name)** — Alphabetic routing: names starting with A–M go to worker-1; N–Z go to worker-2. Only one worker is contacted per call.
- **getbylocation(location)** — Broadcast: the master queries *both* workers in sequence and merges their result lists.
- **getbyyear(location, year)** — Broadcast with dual filter: both workers are queried; only records matching both location AND year are returned.

Worker calls are wrapped in `_safe_worker_call()` which catches all network/RPC exceptions. If a worker is unreachable the master still returns whatever partial results it collected, with `partial: True` and a descriptive message — the client is never left with an unhandled exception.

4. Dataset

Each worker loads a JSON file at startup. Both files share the same schema: **record_id**, **name**, **location**, and **year**. Together they cover 28 unique person records across cities including Kansas City, New York City, Los Angeles, Newport Beach, Miami, Chicago, Saint Louis, Orlando, and Overland Park.

record_id	name	location	year	worker
2	alice	New York City	2002	am
4	bill	Kansas City	2002	am
9	xander	Los Angeles	2005	nz
1	rakin	Kansas City	2019	nz
3	zen	Kansas City	2018	nz
14	niko	Kansas City	2001	nz
...	...	...	...	...

Table 1 — Sample records from both worker datasets

Distributed Master-Worker System

Execution Results & Fault Tolerance Testing

5. System Startup (Screenshot 1 Observation)

The system was started in three separate terminal windows in the following order:

- Worker-1: `python worker.py 23003 am` → loaded 14 records, listening on port 23003
- Worker-2: `python worker.py 23004 nz` → loaded 14 records, listening on port 23004
- Master: `python master.py 23001` → listening on port 23001
- Client: `python client.py 23001` → executed 3 queries

6. Normal Operation Query Results (Both Workers Online)

The client issued three queries. All workers were reachable; results were complete (**partial: False**).

Query 1 — `getbyname('xander')`

```
{'error': False, 'result': [{'record_id': 9, 'name': 'xander', 'location': 'Los Angeles', 'year': 2005}], 'partial': False}
```

Analysis: Name 'xander' starts with 'x' → routed to worker-2 (nz). Single exact match returned.

Query 2 — `getbylocation('Kansas City')`

```
{'error': False, 'result': [ {'record_id': 4, 'name': 'bill', 'location': 'Kansas City', 'year': 2002}, {'record_id': 12, 'name': 'deep', 'location': 'Kansas City', 'year': 2022}, {'record_id': 1, 'name': 'rakin', 'location': 'Kansas City', 'year': 2019}, {'record_id': 3, 'name': 'zen', 'location': 'Kansas City', 'year': 2018}, {'record_id': 14, 'name': 'niko', 'location': 'Kansas City', 'year': 2001}, {'record_id': 4, 'name': 'symon', 'location': 'Kansas City', 'year': 2015}], 'partial': False}
```

Analysis: Broadcast to both workers. 2 matches from worker-1 (am) + 4 from worker-2 (nz) merged correctly.

Query 3 — `getbyyear('New York City', 2002)`

```
{'error': False, 'result': [ {'record_id': 2, 'name': 'alice', 'location': 'New York City', 'year': 2002}], 'partial': False}
```

Analysis: Dual filter broadcast. Only 'alice' from worker-1 matched both location AND year 2002.

7. Fault Tolerance Testing (Screenshots 2 & 3)

To test fault tolerance, one worker was terminated mid-session while the client repeated the same three queries. The master's `_safe_worker_call()` caught the **WinError 10061** (connection actively refused) exception and continued with results from the surviving worker.

7.1 Worker-1 (port 23003) killed

```
getbyname('xander'): {'error': True, 'result': [], 'partial': True, 'message': 'worker-2 failed: [WinError 10061] No connection could be made because the target machine actively refused it'}
```

Since 'xander' routes to worker-2 (nz) but worker-1 was killed, the error here implies a second run where worker-2 was also down. When only worker-1 is down, `getbyname('xander')` still succeeds because it routes exclusively to worker-2.

7.2 Worker-2 (port 23004) killed

```
getbylocation('Kansas City'): {'error': False, 'result': [{'record_id': 4, 'name': 'bill', ...},
{'record_id': 12, 'name': 'deep', ...}], 'partial': True, 'message': 'One or more workers failed:
worker-2: [WinError 10061] No connection could be made because the target machine actively refused
it'}
```

The master returned the 2 partial results from the surviving worker-1, clearly flagged with **partial: True** so the client knows the result set is incomplete. The system never crashed or raised an unhandled exception — it degraded gracefully.

8. Key Implementation Observations

- **Alphabetic Sharding** — Names A–M go to worker-1; N–Z to worker-2. This simple hash keeps each worker's responsibility clear and avoids cross-worker coordination for name lookups.
- **Broadcast Aggregation** — Location and year queries fan out to both workers and merge results, ensuring complete answers regardless of which worker holds a record.
- **Graceful Degradation** — Network failures are caught per-worker; surviving results are always returned with a partial flag rather than raising an error.
- **Stateless Workers** — Workers load their JSON dataset once at startup and serve read-only queries. No shared mutable state means no synchronisation overhead.
- **Zero External Dependencies** — The entire system uses Python's standard library (xmlrpc.server, xmlrpc.client, json), making it immediately runnable without pip installs.